

Indian Institute of Information Technology

Surat

Department of CSE

Odd Semester Endsem Exam 2024-25

BTech: Semester III

Subject code: (CS-301)

Duration: 03 Hours

Roll Number :

U 1 2 4 C S 6

Subject: Operating System

Date: 2nd December, 2025

Marks: 100

Note

1. All questions are compulsory
2. Uses of scientific and programmable calculators are strictly prohibited
3. Assume the data wherever required
4. Write your roll number compulsorily on the question paper

Q. No.	Description	Marks
1	a. Suppose that a CPU scheduling algorithm favors those processes that have used the least processor time in the recent past. Why will this algorithm favor I/O-bound programs and yet not permanently starve CPU-bound programs? Use a diagram for explanation. b. What advantage is there in having different time-quantum sizes at different levels of a multilevel queueing system? Explain with an example and diagram. c. Race conditions are possible in many computer systems. Consider an online auction system where the current highest bid for each item must be maintained. A person who wishes to bid on an item calls the bid(amount) function, which compares the amount being bid to the current highest bid. If the amount exceeds the current highest bid, the highest bid is set to the new amount. This is illustrated below: <pre>void bid(double amount) { if (amount > highestBid) highestBid = amount; }</pre> Describe how a race condition is possible in this situation and what might be done to prevent the race condition from occurring.	05

	d	Describe how the signal() operation associated with monitors differs from the corresponding operation defined for semaphores.	05																												
	e	Can a system detect that some of its threads are starving? If you answer "yes," explain how it can. If you answer "no," explain how the system can deal with the starvation problem.	05																												
2	a	Show that, if the wait() and signal() semaphore operations are not executed atomically, then mutual exclusion may be violated.	05																												
	b	Consider the following snapshot of a system: <table border="1"> <thead> <tr> <th></th> <th>Allocation</th> <th>Max</th> <th>Available</th> </tr> <tr> <th></th> <th>A B C D</th> <th>A B C D</th> <th>A B C D</th> </tr> </thead> <tbody> <tr> <td>T₀</td> <td>0 0 1 2</td> <td>0 0 1 2</td> <td>1 5 2 0</td> </tr> <tr> <td>T₁</td> <td>1 0 0 0</td> <td>1 7 5 0</td> <td></td> </tr> <tr> <td>T₂</td> <td>1 3 5 4</td> <td>2 3 5 6</td> <td></td> </tr> <tr> <td>T₃</td> <td>0 6 3 2</td> <td>0 6 5 2</td> <td></td> </tr> <tr> <td>T₄</td> <td>0 0 1 4</td> <td>0 6 5 6</td> <td></td> </tr> </tbody> </table> Answer the following questions using the banker's algorithm: - What is the content of the matrix Need? - Is the system in a safe state? - If a request from thread T₁ arrives for (0, 4, 2, 0), can the request be granted immediately?		Allocation	Max	Available		A B C D	A B C D	A B C D	T ₀	0 0 1 2	0 0 1 2	1 5 2 0	T ₁	1 0 0 0	1 7 5 0		T ₂	1 3 5 4	2 3 5 6		T ₃	0 6 3 2	0 6 5 2		T ₄	0 0 1 4	0 6 5 6		05
	Allocation	Max	Available																												
	A B C D	A B C D	A B C D																												
T ₀	0 0 1 2	0 0 1 2	1 5 2 0																												
T ₁	1 0 0 0	1 7 5 0																													
T ₂	1 3 5 4	2 3 5 6																													
T ₃	0 6 3 2	0 6 5 2																													
T ₄	0 0 1 4	0 6 5 6																													
	c	Consider a logical address space of 64 pages of 1,024 words each, mapped onto a physical memory of 32 frames. - How many bits are there in the logical address? - How many bits are there in the physical address?	05																												
	d	Given six memory partitions of 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, and 125 KB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 115 KB, 500 KB, 358 KB, 200 KB, and 375 KB (in order)?	05																												
	e	An operating system has a 21-bit virtual address, yet on certain embedded devices, it has only a 16-bit physical address. It also has a 2-KB page size. How many entries are there in each of the following? - A conventional, single-level page table - An inverted page table - What is the maximum amount of physical memory in the 32V operating system?	05																												
3	a	A semaphore-based solution to a synchronisation problem uses the following initial values: S ₁ = 1, S ₂ = 0, S ₃ = 0.	5																												

		P1:	P2:	P3:	
		wait(S1) print("A") signal(S2)	wait(S2) print("B") signal(S2)	wait(S3) print("C") signal(S1)	
		a. What will be the output order? b. Could any process block forever? Explain.			
	b	Paging removes external fragmentation by allowing non-contiguous allocation, but internal fragmentation can still occur. Explain this with an example, and discuss how the choice of page size affects the amount of internal fragmentation.			05
	c	A running process references a virtual page that is not currently resident in physical memory. Explain how the operating system handles this page fault, including the steps of fault detection, locating and retrieving the required page, managing frame allocation or replacement, and resuming the process. Briefly discuss the performance implications of this sequence of operations.			05
	d	Consider a process with five user-level threads where the mapping of user threads to kernel threads is many-to-one. If one user thread incurs a page fault while accessing its stack, would the other user threads of the same process also be affected, i.e., would they have to wait for the faulting page to be brought into memory? Explain your answer.			05
	e	Explain why thrashing occurs in a multiprogrammed system. Illustrate with an example how excessive page faults can reduce CPU utilization. How can the operating system detect thrashing, and what strategies can it use to prevent or control it?			05
4	a	Consider a paging system in which the page table is stored entirely in main memory. Assume that one memory access requires 50 nanoseconds. a. If the system does not use a Translation Lookaside Buffer (TLB), calculate the total time required to service a single memory reference. Explain why two memory accesses are needed. b. Now assume that the system uses a TLB. If 75% of the page-table lookups result in a TLB hit and the time required to search the TLB is negligible compared to memory access time, compute the effective memory access time. Show all steps clearly. c. Briefly discuss how increasing the TLB hit ratio affects			10

		overall system performance, and mention two factors that influence the hit ratio.	
	b	Consider the following page reference string: 7, 2, 3, 1, 2, 5, 3, 4, 6, 7, 7, 1, 0, 5, 4, 6, 2, 3, 0, 1 Assuming demand paging, analyse the number of page faults produced by the following replacement algorithms: 1. FIFO (First-In, First-Out) 2. LRU (Least Recently Used) 3. Optimal Show all workings (frame tables). a. Perform the analysis for 3 frames, showing frame contents after each reference and total page faults for each algorithm. b. Repeat the analysis for 4 frames. Show frame contents and total page faults for each algorithm. c. Briefly compare the results: comment on how the algorithms differ in page-fault rates and how increasing the number of frames affects performance.	10
	c	A disk drive has 5,000 cylinders numbered 0 to 4,999. The disk head is currently at cylinder 2,150. The queue of pending requests in FIFO order is: 2,069; 1,212; 2,296; 2,800; 544; 1,618; 356; 1,523; 4,965; 3,681 Calculate the total distance (in cylinders) moved by the disk arm to service all requests using the FCFS (First-Come, First-Served) scheduling algorithm. OR A program executes a loop 100,000 times, and during each iteration it accesses exactly two pages: page 5 and page 6. The system follows pure demand paging, and both pages are initially not in memory. 1. How many page faults occur after the first iteration of the loop? 2. Explain why pure demand paging is highly efficient for programs with repeated access patterns such as loops.	05